

DataSoul AI

The Complete Technical Deep-Dive

A Comprehensive Tutorial on Building an
Intelligent Data Cleaning Engine

~7,500 lines of Python across 14 modules

FastAPI | Pandas | NumPy | Scikit-learn | ChromaDB | Ollama

Generated from Source Code Analysis

DataSoul AI: The Complete Technical Deep-Dive

A Comprehensive Tutorial on Building an Intelligent Data Cleaning Engine

Author: Generated from Source Code Analysis Project: DataSoul AI -- Intelligent Data Cleaning & Analysis Engine

Codebase: ~7,500 lines of Python across 14 modules Stack: FastAPI | Pandas | NumPy | Scikit-learn | ChromaDB
| Ollama

PART I: INTRODUCTION & ARCHITECTURE OVERVIEW

Chapter 1: What is DataSoul AI?

DataSoul AI is a full-stack, AI-augmented data cleaning and analysis engine built in Python. It accepts messy, real-world CSV and Excel files, automatically identifies quality issues, cleans the data using a combination of deterministic algorithms and Large Language Model (LLM) intelligence, and produces executive-grade reports about the dataset's health.

The Problem DataSoul Solves

Every data scientist spends 60-80% of their time on data cleaning. The typical workflow looks like this:

- * 1. Load a CSV into Pandas.
- * 2. Run `df.info()` and `df.describe()` to get an overview.
- * 3. Manually inspect columns for missing values, duplicates, encoding issues.
- * 4. Write one-off cleaning scripts: strip whitespace, fix types, handle nulls.
- * 5. Re-check the data. Repeat steps 3-4 many times.
- * 6. Eventually give up and hope the remaining issues don't affect the model.

DataSoul automates this entire workflow. But it does something no pure-Python tool can do: it uses a local LLM to discover patterns that rule-based systems miss. The LLM examines sample values and identifies dirty patterns like "8GB" (a unit suffix embedded in a number), "Writtenby:J.K. Rowling" (a label prefix contaminating a name), or "4.5 out of 5 stars 234 ratings" (a rating string that should be a float). Once the LLM identifies WHAT to clean, standard Python code executes the fix at scale.

How DataSoul Differs from Existing Tools

Feature	Pandas Profiling	Great Expectations	DataSoul AI
Profiling	Yes	No	Yes
Quality Rules	No	Yes (manual)	Yes (auto-detected)
Auto-Cleaning	No	No	Yes
LLM Intelligence	No	No	Yes
RAG Knowledge Base	No	No	Yes
Executive Narratives	No	No	Yes
ML Predictions	No	No	Yes
Audit Trail	No	Yes	Yes

Works Offline	Yes	Yes	Yes
---------------	-----	-----	-----

The "Vibecoded" Development Philosophy

This project was "vibecoded" -- built through rapid, AI-assisted development where the developer describes what they want and an AI coding assistant helps implement it. This is not a criticism; it is a legitimate development methodology that emphasizes:

- * 1. Speed of iteration over premature optimization.
- * 2. Working software over comprehensive documentation (which this tutorial retroactively provides).
- * 3. Pragmatic architecture over theoretical purity.
- * 4. Graceful degradation over hard dependencies.

The result is a system that is structurally sound, functionally complete, and serves as an excellent educational case study in how modern AI-augmented Python applications are built.

Chapter 2: System Architecture -- The 30,000-Foot View

DataSoul's backend consists of 14 Python modules organized into four functional domains. Here is the complete architecture:

```

+-----+
|  React Frontend  |
| (Next.js / UI)  |
+-----+-----+
|
| HTTP/REST
|
+-----V-----+
|  main.py        | <-- API Orchestrator (FastAPI)
|  1,191 lines    |   Receives all requests, routes
+-----+-----+   to the correct engine
| | | | |
+-----+ | | +-----+
| | | | |
+-----V-----+ +-----V--V--V-----+ +-----V-----+
| INTELLIGENCE | | THE ENGINE | | PRESENTATION |
| LAYER        | | (Cleaning) | | LAYER        |
+-----+-----+ +-----+-----+ +-----+-----+
| llm_engine   | | profiler.py  | | narrative_eng |
| (Ollama REST)| | threat_detector.py | | prediction_eng |
| rag_engine   | | data_cleaners.py | | strategy_eng  |
| (ChromaDB)  | | llm_cleaner.py  | | sector_detect  |
| knowledge_   | | csv_corrector.py | +-----+-----+
| scraper      | | pipeline_engine.py |
+-----+-----+ +-----+-----+

```

Domain 1: Infrastructure Layer

- * `main.py` (1,191 lines): The FastAPI application. Every HTTP request enters here. It manages global dataset state, routes requests to the correct engine, and returns JSON responses. Think of it as the receptionist of a hospital -- it doesn't perform surgery, but nothing happens without it.
- * `llm_engine.py` (562 lines): The REST client for Ollama, a local LLM runtime. This module handles all communication with the AI model: sending prompts, receiving responses, streaming output, and managing model availability. It is the "telephone line" to the AI brain.

Domain 2: The Brain (Knowledge System)

- * `rag_engine.py` (681 lines): The Retrieval-Augmented Generation engine built on ChromaDB. It stores and retrieves domain knowledge, making the LLM contextually aware of data science best practices and the current dataset's characteristics.
- * `knowledge_scraper.py` (624 lines): Populates the RAG knowledge base by scraping documentation (scikit-learn, Pandas, Wikipedia) and providing a massive built-in knowledge corpus of 200+ curated entries covering imputation, encoding, scaling, sector-specific analysis, and data governance.

Domain 3: The Engine (Data Processing)

- * `profiler.py` (344 lines): Generates comprehensive dataset profiles: statistics, distributions, outlier detection, and a 7-dimension quality score (Completeness, Uniqueness, Consistency, Type Correctness, Validity, Accuracy, Timeliness).
- * `threat_detector.py` (308 lines): Identifies data quality threats across 8 categories: missing values, duplicates, inconsistencies, mixed types, outliers, ML risks, privacy/PII, and business risks.
- * `data_cleaners.py` (783 lines): Pure deterministic cleaning utilities. 12 specialized cleaners for null strings, whitespace, percentages, booleans, dates, phone numbers, emails, and more. No LLM required -- these are fast, reliable, and testable.
- * `llm_cleaner.py` (598 lines): The LLM-guided cleaning layer. The LLM analyzes column samples and produces a JSON cleaning plan. Python then executes each plan deterministically. This is the "LLM for intelligence, code for execution" pattern in its purest form.
- * `csv_corrector.py` (881 lines): The full correction orchestrator. Handles encoding repair (mojibake), type coercion, category merging (exact + fuzzy + LLM), and value correction. Combines deterministic and LLM-guided approaches.
- * `pipeline_engine.py` (810 lines): The iterative pipeline that orchestrates the entire cleaning process: profile, clean, re-profile, compare, and repeat.

Domain 4: Presentation Layer

- * `narrative_engine.py` (422 lines): Generates executive reports, computes KPIs, and powers the conversational data chat ("Ask AI") feature.
- * `prediction_engine.py` (428 lines): ML-powered predictions using scikit-learn (RandomForest, IsolationForest) with optional LLM interpretation of results.

- * strategy_engine.py (220 lines): Recommends preprocessing strategies (imputation, encoding, scaling) based on data characteristics.
- * sector_detector.py (130 lines): Automatically detects the business sector (Retail, Healthcare, Finance, HR, etc.) from column names using weighted pattern matching.

The Data Flow

The complete lifecycle of a dataset through DataSoul follows this path:

```

User Uploads CSV
|
v
[1] PARSE & STORE --> datasets[key] = {"df": DataFrame, ...}
|
v
[2] PROFILE -----> DataProfiler.generate_full_profile()
|                    - Column stats, distributions, quality score
v
[3] DETECT THREATS ---> ThreatDetector.detect_all_threats()
|                    - Missing values, PII, outliers, ML risks
v
[4] DETECT SECTOR ----> SectorDetector.detect()
|                    - Industry classification for context
v
[5] INDEX IN RAG -----> RAGEngine.index_dataset_context()
|                    - Embed column names, samples for retrieval
v
[6] CLEAN (Pipeline) -> IterativePipeline.run()
|                    Stage A: DataCleaners.run_all()
|                    Stage B: LLMCleaner.clean_async()
|                    Stage C: CSVCorrector.auto_correct()
v
[7] RE-PROFILE -----> Compare before/after quality scores
|
v
[8] GENERATE REPORT --> NarrativeEngine.generate_story()
|
v
[9] EXPORT -----> CSV / Excel / JSON / Jupyter Notebook

```

Chapter 3: The Design Philosophy -- "LLM for Intelligence, Code for Execution"

This is the single most important architectural decision in DataSoul, and understanding it is key to understanding every module in the codebase.

The Problem with Pure LLM Approaches

Imagine you have a CSV with 50,000 rows and a column containing values like "8GB", "16GB", "4GB". A naive approach would be to send each value to the LLM and ask it to extract the number. This fails for three reasons:

- * 1. Speed: 50,000 LLM calls at ~500ms each = 7 hours. Unacceptable.
- * 2. Cost: Even with local models, the GPU compute is wasted on repetitive work.
- * 3. Consistency: The LLM might return "8" for one row and "8.0" for another, or hallucinate entirely.

The Problem with Pure Deterministic Approaches

A rule-based system can strip "GB" from the end of strings. But what about:

- * "1.37kg" (weight with unit suffix)
- * "4.5 hrs" vs "270 min" (different time units)
- * "Writtenby:J.K. Rowling" (a label prefix from web scraping)
- * "4.5 out of 5 stars 234 ratings" (a complex rating string)

Writing regex rules for every possible pattern is a losing battle. New datasets bring new patterns that the rules have never seen.

DataSoul's Hybrid Solution

DataSoul splits the problem into two phases:

Phase 1 -- Discovery (LLM): Send a sample of ~15 values from each column to the LLM. Ask it: "What dirty patterns do you see? How should they be cleaned?" The LLM returns a structured JSON plan:

```
{
  "RAM": [
    {
      "issue": "unit_suffix",
      "fix_type": "extract_number",
      "extract_regex": "([\\d.]+)\\s*GB",
      "confidence": 0.95
    }
  ]
}
```

Phase 2 -- Execution (Python): Take the LLM's plan and execute it with Pandas at full speed:

```
df["RAM"] = df["RAM"].str.extract(r"([\\d.]+)\\s*GB").astype(float)
```

This approach gives you the best of both worlds:

- * The LLM's pattern recognition (1 call per column, not per row)
- * Python's execution speed (vectorized Pandas operations on all 50,000 rows)
- * Deterministic consistency (the same regex is applied to every row)
- * Full auditability (the plan is logged, not a black-box transformation)

Graceful Degradation

Every module in DataSoul follows a strict rule: the system must work even without Ollama running. This is implemented through the `is_available` property pattern:

```
class LLMCleaner:
    @property
    def is_available(self) -> bool:
        return self._llm is not None and self._llm.is_available

    async def clean_async(self, df, min_confidence=0.75):
        if not self.is_available:
            return df, [] # <-- Return unchanged data, empty audit
        # ... LLM-powered cleaning logic ...
```

When Ollama is offline:

- * `data_cleaners.py` still runs all 12 deterministic cleaners (null strings, whitespace, types, etc.)
- * `csv_corrector.py` still fixes encoding, merges case-variant categories, and coerces types
- * `profiler.py` and `threat_detector.py` still produce full profiles and threat reports
- * `narrative_engine.py` falls back to template-based reports instead of LLM-generated narratives
- * `prediction_engine.py` falls back to statistical imputation (median/mode) instead of ML models

The system degrades from "intelligent" to "competent" -- never to "broken."

The Confidence Scoring System

Every cleaning action in DataSoul carries a confidence score from 0 to 100:

Range	Meaning	Action
90-100	Near-certain correctness	Auto-apply without user approval
75-89	High confidence	Auto-apply with audit trail entry
60-74	Moderate confidence	Suggest to user, don't auto-apply
Below 60	Low confidence	Log but do not suggest

This system is implemented in `csv_corrector.py` via two thresholds:

```
class CSVCorrector:
    AUTO_APPLY_THRESHOLD = 0.90 # Auto-apply if confidence >= 90%
    SUGGEST_THRESHOLD = 0.60 # Show to user if confidence >= 60%
```

And in `llm_cleaner.py` via the `min_confidence` parameter:

```
def clean(self, df, min_confidence=0.75):
    # Only apply plans where confidence >= 0.75
```

The Audit Trail Contract

Every cleaning function in the entire system returns an audit dictionary with this structure:

```
{
  "action": "CLEAN_NULL_STRINGS",      # What was done
  "column": "Status",                 # Which column was modified
  "detail": "Converted 42 null-like strings ('N/A', 'null', '-') to NaN",
  "rows_affected": 42,                # Exactly how many values changed
  "confidence": 96,                   # How sure the system is (0-100)
  "source": "deterministic",          # "deterministic" or "LLM-guided"
}
```

This contract is enforced across all three cleaning layers:

- * data_cleaners.py returns (cleaned_series, audit_dict) tuples
- * llm_cleaner.py uses the _audit() helper function
- * csv_corrector.py builds audit lists at every stage

The audit trail flows upward: individual cleaners -> CSVCorrector -> Pipeline -> API response -> Frontend display. This gives the user complete visibility into exactly what was changed, why, and how confident the system was.

Chapter 4: Getting Started -- Environment & Dependencies

The Technology Stack

DataSoul's requirements.txt specifies these dependencies:

```
fastapi
uvicorn[standard]
pandas
numpy
scikit-learn
chromadb
httpx
requests
python-multipart
openpyxl
aiofiles
```

Here is what each dependency does and WHY it is needed:

Package	Purpose	Why Not an Alternative?
fastapi	Web framework for the REST API	Async-native, automatic OpenAPI docs, type validation via Pydantic
uvicorn	ASGI server to run FastAPI	The standard production server for FastAPI
pandas	DataFrame operations, CSV parsing	Industry standard for tabular data in Python

numpy	Numerical operations, statistics	Underlies Pandas; used for polyfit, quantiles
scikit-learn	ML models (RandomForest, IsolationForest, ...)	Most mature ML library in Python
chromadb	Vector database for RAG	Lightweight, embeddable, no server needed
httpx	HTTP client for Ollama API calls	Async-capable, modern alternative to requests
requests	HTTP client for web scraping	Simpler API for one-shot scraping calls
python-multipart	File upload parsing	Required by FastAPI for multipart form data
openpyxl	Excel file reading/writing	Pandas uses this engine for .xlsx files
aiofiles	Async file I/O	Non-blocking file operations in async routes

External Services

Ollama (optional but recommended): A local LLM runtime that must be running on port 11434. Install from <https://ollama.com> and start with `ollama serve`. DataSoul will automatically pull a compatible model on first use.

Models are tried in this order (defined in `llm_engine.py`):

```
FALLBACK_MODELS = [
    "qwen2.5:7b",
    "qwen2.5-coder:7b",
    "llama3.2:3b",
    "llama3.1:8b",
    "mistral:7b",
    "gemma2:9b",
    "phi3:mini",
]
```

Directory Structure

```

DataSoul/
backend/
  main.py                # API entry point
  llm_engine.py          # Ollama REST client
  rag_engine.py          # ChromaDB vector store
  knowledge_scraper.py   # Knowledge base builder
  profiler.py            # Data quality profiling
  threat_detector.py     # Threat detection
  data_cleaners.py       # Deterministic cleaners
  llm_cleaner.py         # LLM-guided cleaning
  csv_corrector.py       # Full correction engine
  pipeline_engine.py     # Iterative pipeline
  narrative_engine.py    # Report generation
  prediction_engine.py   # ML predictions
  strategy_engine.py     # Preprocessing recommendations
  sector_detector.py     # Industry detection
  integrations/          # External data source connectors
  uploads/              # Uploaded file storage
  rag_store/             # ChromaDB persistence directory
  requirements.txt
datasoul_brain/         # RAG knowledge documents
  knowledge_base/        # Scraped + built-in knowledge chunks
frontend/               # Next.js React application

```

Starting the Backend

```

cd backend
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000

```

On first boot, the following happens:

- * 1. FastAPI initializes and registers all routes.
 - * 2. `get_llm()` creates the singleton `LLMEngine`, which checks if Ollama is running.
 - * 3. `get_rag()` creates the singleton `RAGEngine`, which initializes ChromaDB and ingests the `datasoul_brain/` directory.
 - * 4. The system is ready to accept requests at `http://localhost:8000`.
-

PART II: THE API ORCHESTRATION LAYER --

main.py

Chapter 5: FastAPI Foundation & Application Setup

The file `main.py` is the central nervous system of DataSoul. At 1,191 lines, it is the largest file in the codebase, but its role is clear: receive HTTP requests, coordinate the backend engines, and return responses.

Application Initialization

```
from fastapi import FastAPI, UploadFile, File, Form, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
import asyncio, pandas as pd, numpy as np, json, os, io, tempfile

app = FastAPI(title="DataSoul AI", version="2.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Why CORS? The frontend (a Next.js React app) runs on `localhost:3000`, while the backend runs on `localhost:8000`. Browsers block cross-origin requests by default for security. The CORS middleware tells the browser: "Yes, the frontend at any origin is allowed to call my API." In production, you would restrict `allow_origins` to your specific domain.

Global State Management

DataSoul uses a simple but effective pattern for managing datasets in memory:

```
datasets: dict = {}
current_key: str | None = None
```

The `datasets` dictionary maps string keys (derived from filenames) to dataset records:

```
datasets["sales_data.csv"] = {
    "df": pd.DataFrame(...),          # The actual data
    "original_df": pd.DataFrame(...), # Untouched copy for comparison
    "profile": {...},                 # Cached profiling results
    "threats": {...},                 # Cached threat detection
    "sector": {...},                  # Detected business sector
    "audit": [...],                   # Cumulative cleaning audit trail
}
```

This design means:

- * Multiple datasets can be loaded simultaneously.
- * Profiling results are cached so you do not re-profile on every request.
- * The original DataFrame is preserved for before/after comparison.
- * The audit trail grows as cleaning operations are applied.

The `current_key` tracks which dataset is "active." Most endpoints operate on the current dataset.

The `\_df()` Helper Pattern

A small but important helper appears throughout `main.py`:

```
def _df() -> pd.DataFrame:
    if not current_key or current_key not in datasets:
        raise HTTPException(status_code=400, detail="No dataset loaded")
    return datasets[current_key]["df"]
```

This pattern provides:

- * 1. Safe access: If no dataset is loaded, the user gets a clear error instead of a cryptic `KeyError`.
- * 2. DRY principle: Every endpoint that needs the current DataFrame calls `_df()` instead of repeating the check.

The Async Architecture

One of the most critical architectural decisions in `main.py` is how it handles CPU-intensive operations. FastAPI is an async framework built on Python's `asyncio` event loop. If a route handler does heavy computation (like profiling a 100,000-row DataFrame), it blocks the event loop and no other requests can be served.

DataSoul solves this with `asyncio.to_thread`:

```
@app.post("/profile")
async def profile_dataset():
    df = _df()
    result = await asyncio.to_thread(_run_profiling, df)
    return result

def _run_profiling(df):
    # This runs in a separate thread, not blocking the event loop
    profiler = DataProfiler(df)
    return profiler.generate_full_profile()
```

Why this matters: Without `asyncio.to_thread`, a profiling request on a large dataset would block the server for 5-10 seconds. During that time, the frontend's health check pings would time out, and the UI would show "Engine Unavailable." By offloading to a thread, the event loop stays responsive.

This pattern is used for ALL heavy operations: profiling, cleaning, pipeline runs, corrections, and predictions.

Chapter 6: Data Ingestion Endpoints

DataSoul supports five data ingestion methods:

1. CSV/Excel Upload (`POST /upload`)

The primary ingestion endpoint accepts file uploads:

```

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    contents = await file.read()
    filename = file.filename

    # Detect file type and parse accordingly
    if filename.endswith((".xlsx", ".xls")):
        df = pd.read_excel(io.BytesIO(contents))
    else:
        # Try multiple encodings for CSV
        for encoding in ["utf-8", "latin-1", "cp1252"]:
            try:
                df = pd.read_csv(io.BytesIO(contents), encoding=encoding)
                break
            except UnicodeDecodeError:
                continue

    # Store in global state
    key = filename
    datasets[key] = {
        "df": df,
        "original_df": df.copy(),
        "profile": None,
        "threats": None,
        "audit": [],
    }
    current_key = key

    # Trigger initial profiling
    profile = await asyncio.to_thread(lambda: DataProfiler(df).generate_full_profile())
    datasets[key]["profile"] = profile

    return {"status": "success", "rows": len(df), "columns": len(df.columns), ...}

```

Key design decisions:

- * Multi-encoding fallback: Many real-world CSVs are not UTF-8. The code tries UTF-8 first, then Latin-1 (Western European), then CP1252 (Windows). This handles 99% of CSVs without the user needing to specify encoding.
- * Immediate profiling: The upload triggers an initial profile so the frontend can immediately show data quality metrics.
- * Original copy: `df.copy()` preserves the pristine data for before/after comparison.

2. Kaggle Import (`POST /import/kaggle`)

Downloads datasets directly from Kaggle using the kaggle CLI tool:

```
@app.post("/import/kaggle")
async def import_kaggle(dataset_id: str = Form(...)):
    # Uses subprocess to call: kaggle datasets download -d <id> -p uploads/
    result = await asyncio.to_thread(_download_kaggle, dataset_id)
    return result
```

The implementation calls kaggle datasets download as a subprocess, extracts the downloaded ZIP file, and loads the CSV. This requires the user to have Kaggle API credentials configured (~/.kaggle/kaggle.json).

3. Google Sheets Import (`POST /import/google-sheets`)

Loads data from public Google Sheets by converting the share URL to a CSV export URL:

```
# Convert: https://docs.google.com/spreadsheets/d/SHEET_ID/edit
# To:      https://docs.google.com/spreadsheets/d/SHEET_ID/export?format=csv
```

4. Data.gov.in Import (`POST /import/data-gov-in`)

Connects to India's Open Data Portal API to download government datasets.

5. SQL Import (`POST /import/sql`)

Connects to SQL databases using pandas.read_sql with a user-provided connection string and query.

Chapter 7: Profiling & Analysis Endpoints

`POST /profile` -- Full Dataset Profiling

This endpoint triggers the complete profiling pipeline:

```

@app.post("/profile")
async def profile_dataset():
    df = _df()
    key = current_key

    # Run profiling in a background thread
    profile = await asyncio.to_thread(lambda: DataProfiler(df).generate_full_profile())

    # Run threat detection using the profile
    threats = await asyncio.to_thread(
        lambda: ThreatDetector(df, profile).detect_all_threats()
    )

    # Detect business sector
    sector = SectorDetector().detect(df)

    # Index dataset context in RAG for future LLM queries
    rag = get_rag()
    if rag.is_ready:
        rag.index_dataset_context(df, profile)

    # Cache results
    datasets[key]["profile"] = profile
    datasets[key]["threats"] = threats
    datasets[key]["sector"] = sector

    return {"profile": profile, "threats": threats, "sector": sector}

```

This single endpoint triggers FOUR operations:

- * 1. DataProfiler: Statistical analysis of every column.
- * 2. ThreatDetector: Identifies data quality risks from the profile.
- * 3. SectorDetector: Classifies the business domain.
- * 4. RAGEngine: Indexes dataset metadata for context-aware LLM queries.

Results are cached in the datasets dictionary so subsequent requests do not re-compute.

GET /quality-score -- The 7-Dimension Health Score

Returns just the quality score from the cached profile:

```

@app.get("/quality-score")
async def get_quality_score():
    profile = datasets[current_key].get("profile")
    if not profile:
        raise HTTPException(400, "Profile not generated yet")
    return profile.get("quality_score", {})

```

GET /suggest-strategies -- ML Preprocessing Recommendations

Runs the StrategyEngine against the profile:

```
@app.get("/suggest-strategies")
async def suggest_strategies():
    profile = datasets[current_key].get("profile")
    strategies = StrategyEngine().recommend(_df(), profile)
    return strategies
```

Chapter 8: Cleaning & Correction Endpoints

`POST /correct/auto` -- Full Auto-Correction

Runs the CSVCorrector's complete pipeline:

```
@app.post("/correct/auto")
async def auto_correct():
    df = _df()
    profile = datasets[current_key].get("profile")

    result = await asyncio.to_thread(
        lambda: CSVCorrector().auto_correct(df, profile=profile)
    )

    if result["status"] == "success":
        datasets[current_key]["df"] = result["df"]
        datasets[current_key]["audit"].extend(result["audit"])
        # Invalidate cached profile (data changed)
        datasets[current_key]["profile"] = None

    return result
```

Key pattern: After modifying the DataFrame, the cached profile is invalidated (None). This forces a re-profile on the next /profile call, ensuring the quality score reflects the current data state.

`POST /pipeline/run` -- Iterative Pipeline

The most powerful endpoint -- runs the full iterative cleaning pipeline:

```

@app.post("/pipeline/run")
async def run_pipeline():
    df = _df()
    pipeline = IterativePipeline(df)
    result = await asyncio.to_thread(pipeline.run)

    datasets[current_key]["df"] = result["df"]
    datasets[current_key]["audit"].extend(result.get("audit", []))
    datasets[current_key]["profile"] = result.get("final_profile")
    datasets[current_key]["threats"] = result.get("final_threats")

    return result

```

POST /clean/llm -- LLM-Only Cleaning

Runs just the LLM-guided cleaning layer:

```

@app.post("/clean/llm")
async def llm_clean():
    df = _df()
    llm = get_llm()
    cleaner = LLMCleaner(llm)

    cleaned_df, audit = await asyncio.to_thread(
        lambda: cleaner.clean(df, min_confidence=0.75)
    )

    datasets[current_key]["df"] = cleaned_df
    datasets[current_key]["audit"].extend(audit)
    return {"status": "success", "actions": len(audit), "audit": audit}

```

Chapter 9: Intelligence & Prediction Endpoints

POST /ask-ai -- Conversational Data Chat

The "Ask AI" feature lets users ask natural language questions about their dataset:

```

@app.post("/ask-ai")
async def ask_ai(question: str = Form(...)):
    df = _df()
    profile = datasets[current_key].get("profile")
    threats = datasets[current_key].get("threats")

    # Get RAG context for the question
    rag = get_rag()
    rag_context = ""
    if rag.is_ready:
        rag_context = rag.get_context_for_prompt(question, n_results=3)

    # Generate answer
    engine = NarrativeEngine()
    answer = await asyncio.to_thread(
        lambda: engine.answer_question(question, df, profile,
                                         rag_context=rag_context, threats=threats)
    )
    return {"answer": answer}

```

The answer generation follows a priority chain:

- * 1. If Ollama is available and the question is complex: use LLM with RAG context.
- * 2. If the question matches a known pattern (missing values, duplicates, etc.): use template.
- * 3. If the question references a column name: show column statistics.
- * 4. If none of the above: return a helpful "try asking..." response.

POST /predict/missing -- ML-Powered Imputation

Uses RandomForest to predict missing values:

```

@app.post("/predict/missing")
async def predict_missing(target_col: str = Form(...)):
    df = _df()
    profile = datasets[current_key].get("profile")
    engine = PredictionEngine()
    result = await asyncio.to_thread(
        lambda: engine.predict_missing(df, target_col, profile=profile)
    )
    return result

```

POST /generate-story -- Executive Narrative

Generates a full executive report:

```

@app.post("/generate-story")
async def generate_story():
    df = _df()
    profile = datasets[current_key].get("profile")
    threats = datasets[current_key].get("threats")

    rag = get_rag()
    rag_context = rag.get_context_for_prompt(
        "executive data analysis narrative", n_results=5
    ) if rag.is_ready else ""

    engine = NarrativeEngine()
    story = await asyncio.to_thread(
        lambda: engine.generate_story(df, profile, threats, rag_context=rag_context)
    )
    return {"story": story}

```

Chapter 10: Export & Utility Endpoints

Multi-Format Export

DataSoul exports data in three formats:

```

@app.get("/export/csv")
async def export_csv():
    df = _df()
    output = io.StringIO()
    df.to_csv(output, index=False)
    return StreamingResponse(io.BytesIO(output.getvalue().encode()),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=cleaned.csv"})

```

Jupyter Notebook Generation (`POST /generate-notebook`)

One of the most creative features -- DataSoul can generate a complete Jupyter notebook documenting the cleaning process:

```

@app.post("/generate-notebook")
async def generate_notebook():
    df = _df()
    audit = datasets[current_key].get("audit", [])
    profile = datasets[current_key].get("profile", {})

    # Generate notebook cells from the audit trail
    cells = [
        {"cell_type": "markdown", "source": "# DataSoul AI - Cleaning Report"},
        {"cell_type": "code", "source": "import pandas as pd\nndf = pd.read_csv('your_data.csv')"},
    ]

    for entry in audit:
        cells.append({
            "cell_type": "markdown",
            "source": f"## {entry['action']}\n{entry['detail']}\n"
                       f"Rows affected: {entry['rows_affected']}, "
                       f"Confidence: {entry['confidence']}%"
        })

    # ... build complete notebook JSON structure ...

```

System Health Monitoring

```

@app.get("/health")
async def health_check():
    llm = get_llm()
    rag = get_rag()
    return {
        "status": "healthy",
        "llm_available": llm.is_available,
        "rag_ready": rag.is_ready,
        "datasets_loaded": len(datasets),
        "current_dataset": current_key,
    }

```

This endpoint is polled by the frontend every few seconds to show real-time engine status.

PART III: THE BRAIN -- RETRIEVAL-AUGMENTED GENERATION

Chapter 11: What is RAG and Why DataSoul Needs It

RAG from First Principles

Imagine you ask a general-purpose LLM: "How should I clean the 'patient_id' column?" The LLM might give generic advice about cleaning string columns. But it does not know:

- * 1. That your dataset is a healthcare dataset.
- * 2. That patient IDs should NEVER have missing values (HIPAA compliance).
- * 3. That the IQR method is inappropriate for ID columns.
- * 4. That your specific column has 3.2% missing values.

Retrieval-Augmented Generation (RAG) solves this by giving the LLM relevant context before it generates a response. The process works in three steps:

```

Step 1: EMBED KNOWLEDGE
"Patient IDs must never have missing values due to HIPAA..."
|
v
Convert text to a numerical vector (embedding)
[0.23, -0.14, 0.87, 0.02, ...] (768 dimensions)
|
v
Store in vector database (ChromaDB)

Step 2: RETRIEVE RELEVANT CONTEXT
User query: "How to handle missing patient_id values?"
|
v
Convert query to embedding vector
|
v
Find nearest neighbors in vector database
|
v
Return top 3 most relevant knowledge chunks

Step 3: AUGMENT LLM PROMPT
"Given this context:
- Patient IDs must never have missing values due to HIPAA...
- Missing IDs break joins, deduplication, and record linkage...
- Cross-reference with source system to recover missing values...

Now answer: How to handle missing patient_id values?"

```

The LLM now gives a specific, contextually aware answer instead of generic advice.

DataSoul's Brain

DataSoul's brain lives in the `datasoul_brain/` directory. It contains curated knowledge about:

- * Data cleaning best practices (imputation, encoding, scaling, outlier handling)
- * Sector-specific knowledge (retail metrics, healthcare compliance, financial fraud patterns)
- * ML pipeline guidance (cross-validation, feature selection, metric selection)
- * Data governance and compliance (GDPR, HIPAA, PII detection)

This knowledge is ingested into ChromaDB, creating a searchable vector index that the LLM can query during cleaning, narrative generation, and question answering.

Chapter 12: The RAG Engine -- `rag_engine.py` (681 lines)

The RAGEngine Class

```
class RAGEngine:
    def __init__(self):
        self._client = chromadb.PersistentClient(path="rag_store")
        self._collection = self._client.get_or_create_collection(
            name="datasoul_knowledge",
            metadata={"hnsw:space": "cosine"}
        )
        self._is_ready = False
```

ChromaDB is an embedded vector database. Unlike PostgreSQL or MongoDB, it runs entirely within your Python process -- no separate server needed. The PersistentClient stores data to disk in the rag_store/ directory, so knowledge persists across restarts.

The hnsw:space: "cosine" setting tells ChromaDB to use cosine similarity for comparing vectors. Cosine similarity measures the angle between two vectors, which is ideal for semantic similarity (two texts about the same topic will have similar embedding angles regardless of length).

The Singleton Pattern

```
_rag_instance: RAGEngine | None = None

def get_rag() -> RAGEngine:
    global _rag_instance
    if _rag_instance is None:
        _rag_instance = RAGEngine()
    return _rag_instance
```

Only one RAGEngine instance exists for the entire application. This ensures:

- * 1. Only one ChromaDB connection is open.
- * 2. All modules share the same knowledge base.
- * 3. No duplicate ingestion occurs.

Ingestion Pipeline

The ingest_directory() method reads files from datasoul_brain/ and loads them into ChromaDB:

```
def ingest_directory(self, directory: str = "datasoul_brain"):
    for filepath in Path(directory).rglob("*"):
        if filepath.suffix in (".md", ".txt", ".json"):
            text = filepath.read_text(encoding="utf-8", errors="ignore")
            chunks = self._chunk_text(text, source=str(filepath))
            self._add_chunks(chunks)
    self._is_ready = True
```

Text Chunking with Overlap: Large documents are split into chunks of ~1,500 characters with 200-character overlap:

```
Document: "AAAA...AAAA BBBB...BBBB CCCC...CCCC"
|--- Chunk 1 ---|
|--- Chunk 2 ---|
|--- Chunk 3 ---|
```

The overlap ensures that if a concept spans a chunk boundary, both adjacent chunks contain enough context. Without overlap, a sentence split across chunks would be incomplete in both.

Dynamic Dataset Indexing

When a new dataset is uploaded, the RAG engine creates embeddings for the dataset's characteristics:

```
def index_dataset_context(self, df, profile):
    # Index column names and sample values
    for col in df.columns:
        sample = df[col].dropna().head(5).tolist()
        text = f"Column '{col}' contains values like: {sample}"
        self._add_chunks([{"text": text, "source": "dataset_context", ...}])

    # Index profile summary
    quality = profile.get("quality_score", {})
    text = f"Dataset has {len(df)} rows, quality score {quality.get('overall', 0)}/100"
    self._add_chunks([{"text": text, "source": "dataset_context", ...}])
```

This is what makes the LLM aware of your specific dataset. When you ask "What's wrong with my data?", the RAG retrieval finds chunks about YOUR columns and YOUR quality score, not generic information.

Context Retrieval

The `get_context_for_prompt()` method retrieves relevant knowledge for a given query:

```
def get_context_for_prompt(self, query, n_results=5, dataset_profile=None):
    results = self._collection.query(
        query_texts=[query],
        n_results=n_results,
    )

    # Format results as context string
    context_parts = []
    for doc in results["documents"][0]:
        context_parts.append(doc)

    return "\n---\n".join(context_parts)
```

The returned context string is injected directly into LLM prompts, giving the model specific, relevant knowledge to work with.

Chapter 13: The Knowledge Scraper -- knowledge_scraper.py (624 lines)

The KnowledgeScraper populates the RAG knowledge base. It has two modes:

Mode 1: Web Scraping

When the requests library is available, it scrapes documentation from:

```
SCRAPE_SOURCES = {
    "sklearn_preprocessing": {
        "urls": [
            "https://scikit-learn.org/stable/modules/preprocessing.html",
            "https://scikit-learn.org/stable/modules/impute.html",
            ...
        ],
        "category": "preprocessing",
    },
    "pandas_docs": {
        "urls": [
            "https://pandas.pydata.org/docs/user_guide/missing_data.html",
            ...
        ],
        "category": "data_manipulation",
    },
    "statistics_reference": {
        "urls": [
            "https://en.wikipedia.org/wiki/Interquartile_range",
            "https://en.wikipedia.org/wiki/Data_cleansing",
            ...
        ],
    },
}
```

The HTML-to-text pipeline uses regex-based stripping (no BeautifulSoup dependency):

```
def _html_to_text(self, html):
    html = re.sub(r"<script[^\>]*>.*?</script>", "", html, flags=re.DOTALL)
    html = re.sub(r"<style[^\>]*>.*?</style>", "", html, flags=re.DOTALL)
    html = re.sub(r"<h([1-6])[^\>]*>(.*?)</h\1>", r"\n## \2\n", html)
    html = re.sub(r"<p[^\>]*>(.*?)</p>", r"\1\n", html, flags=re.DOTALL)
    text = re.sub(r"<[^\>]+>", " ", html) # Strip remaining tags
    return text.strip()
```

Mode 2: Built-in Knowledge Base (Fallback)

The `_generate_builtin_knowledge_chunks()` method contains a massive, curated knowledge base hard-coded directly into the scraper. This ensures the system has expert knowledge even without internet access:

Data Cleaning Patterns (100+ entries):

- * Missing value imputation strategies (mean, median, mode, MICE, KNN)
- * Outlier detection methods (IQR, Z-score, Isolation Forest)
- * Encoding strategies (One-Hot, Label, Target, Frequency)
- * Scaling techniques (Standard, MinMax, Robust)
- * Feature engineering patterns (date decomposition, binning, interactions)

Sector-Specific Knowledge (80+ entries):

- * Retail: CLV, RFM segmentation, market basket analysis, inventory metrics
- * Healthcare: Readmission risk, HIPAA compliance, ICD codes, length of stay
- * Finance: Fraud detection, credit risk, AML patterns
- * HR: Attrition prediction, pay equity, workforce planning
- * Manufacturing: OEE, predictive maintenance, SPC
- * Education: Dropout prediction, learning analytics
- * Logistics: Route optimization, demand forecasting
- * Real Estate: AVM, market analysis metrics

ML Pipeline Best Practices (50+ entries):

- * Cross-validation strategies, train-test splitting, preprocessing pipeline order
- * Data leakage prevention, model selection, hyperparameter tuning
- * Feature importance, metric selection, multicollinearity handling

Data Governance (30+ entries):

- * GDPR, CCPA, HIPAA, FERPA compliance
 - * PII detection and handling
 - * Data retention policies
 - * Audit trail requirements
-

PART IV: THE LLM GATEWAY -- llm_engine.py

Chapter 14: LLM Architecture in DataSoul

What is Ollama?

Ollama is a local LLM runtime. It downloads and runs language models on your own machine, exposing them via a REST API on port 11434. This means:

- * 1. Privacy: Your data never leaves your machine. No API calls to OpenAI or Google.
- * 2. No cost: No per-token billing. Run as many queries as your GPU can handle.
- * 3. Offline capability: Works without internet (after initial model download).
- * 4. Speed: Local inference is faster than cloud API round-trips for small models.

DataSoul communicates with Ollama via HTTP:

```
DataSoul          Ollama
|                  |
|-- POST /api/generate ----->|
| {model: "qwen2.5:7b",      |
|   prompt: "...",          |
|   stream: false}          |
|                             |
|<----- {response: "..."} ----|
```

Model Fallback Chain

The LLMEngine maintains a prioritized list of models:

```
FALLBACK_MODELS = [
    "qwen2.5:7b",           # Best quality for data analysis
    "qwen2.5-coder:7b",     # Good for JSON output
    "llama3.2:3b",          # Smaller, faster fallback
    "llama3.1:8b",          # Meta's latest
    "mistral:7b",           # Good general purpose
    "gemma2:9b",            # Google's model
    "phi3:mini",            # Microsoft's small model
]
```

On initialization, the engine checks which models are available and selects the first one that responds successfully. If no models are installed, it attempts to auto-pull the first model in the list.

Chapter 15: The LLMEngine Class -- Deep Dive (562 lines)

Initialization & Availability

```
class LLMEngine:
    def __init__(self):
        self._base_url = "http://localhost:11434"
        self._client = httpx.Client(timeout=60)
        self._async_client = httpx.AsyncClient(timeout=60)
        self._model = None
        self._available = False
        self._check_availability()

    @property
    def is_available(self) -> bool:
        return self._available and self._model is not None
```

The constructor creates both synchronous (`httpx.Client`) and asynchronous (`httpx.AsyncClient`) HTTP clients. The sync client is used by modules that call the LLM from synchronous contexts. The async client is used for batched parallel operations.

The `_check_availability()` Method

```
def _check_availability(self):
    try:
        resp = self._client.get(f"{self._base_url}/api/tags", timeout=5)
        if resp.status_code == 200:
            models = [m["name"] for m in resp.json().get("models", [])]
            # Find the first available model from our preference list
            for model in FALLBACK_MODELS:
                if any(model in m for m in models):
                    self._model = model
                    self._available = True
                    return
            # No preferred model found -- try to pull one
            self._find_and_pull_model()
    except Exception:
        self._available = False
```

This method:

- * 1. Calls Ollama's `/api/tags` endpoint to list installed models.
- * 2. Searches for a model matching the priority list.
- * 3. If no match, attempts to auto-pull the first model.
- * 4. If Ollama is not running at all, sets `_available = False` gracefully.

Core Generation Methods

Synchronous generation (generate()):

```
def generate(self, prompt, system=None, temperature=0.3,
             max_tokens=500, timeout=30, json_mode=False):
    if not self.is_available:
        return ""

    payload = {
        "model": self._model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": temperature,
            "num_predict": max_tokens,
        },
    }

    if system:
        payload["system"] = system
    if json_mode:
        payload["format"] = "json"

    resp = self._client.post(
        f"{self._base_url}/api/generate",
        json=payload,
        timeout=timeout,
    )
    return resp.json().get("response", "")
```

Async generation (agenerate()):

```
async def agenerate(self, prompt, system=None, temperature=0.3,
                   max_tokens=500, timeout=30, json_mode=False):
    # Same as generate() but uses self._async_client
    resp = await self._async_client.post(
        f"{self._base_url}/api/generate",
        json=payload,
        timeout=timeout,
    )
    return resp.json().get("response", "")
```

The async version is critical for batch operations. When `llm_cleaner.py` needs to analyze 20 columns in batches of 5, it fires 4 concurrent `agenerate()` calls using `asyncio.gather()`. This means all 4 batches run simultaneously instead of sequentially.

The `\_run\_async()` Bridge

This is one of the most subtle and important methods in the entire codebase:

```
def _run_async(self, coro_fn):
    """Bridge sync callers to async LLM methods without deadlocking."""
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        loop = None

    if loop and loop.is_running():
        # We're inside an async context -- run in a new thread
        import concurrent.futures
        with concurrent.futures.ThreadPoolExecutor() as pool:
            future = pool.submit(lambda: asyncio.run(coro_fn()))
            return future.result()
    else:
        # No event loop running -- safe to use asyncio.run()
        return asyncio.run(coro_fn())
```

Why this exists: FastAPI's routes are async, so the event loop is already running when API handlers execute. If a synchronous function (like `CSVCorrector.auto_correct()`) needs to call an async LLM method, it cannot simply call `asyncio.run()` -- that would crash with "cannot run nested event loop." The `_run_async()` bridge detects this situation and runs the async code in a separate thread with its own event loop.

JSON Mode

Many LLM calls in DataSoul request structured JSON output:

```
raw = await self._llm.agenerate(
    prompt,
    system="Return only valid JSON.",
    temperature=0.05,      # Low temperature = deterministic output
    max_tokens=400,
    json_mode=True,        # Tells Ollama to constrain to JSON
)
```

Setting `json_mode=True` adds "format": "json" to the Ollama API request. This instructs the model to output only valid JSON, dramatically reducing parse failures. Combined with `temperature=0.05` (near-deterministic), this produces consistent, parseable results.

Narrative Generation

The `generate_narrative()` method produces executive-quality reports:

```

def generate_narrative(self, profile, threats, df_summary, rag_context=""):
    prompt = f"""Generate an executive data quality report.

DATASET: {df_summary['rows']} rows x {df_summary['cols']} columns
QUALITY SCORE: {profile.get('quality_score', {}).get('overall', 'N/A')}/100
THREATS: {threats.get('total', 0)} total ({threats.get('critical', 0)} critical)

NUMERIC SUMMARY:
{json.dumps(df_summary.get('numeric_summary', {}), indent=2)}

CONTEXT:
{rag_context[:500]}

Write a professional analysis with sections: Executive Summary, Key Findings,
Risk Assessment, and Recommendations."""

    return self.generate(prompt, system="You are a data analyst...",
                        temperature=0.4, max_tokens=1000)

```

The RAG context (rag_context) injects sector-specific knowledge, so a healthcare dataset gets healthcare-aware insights.

PART V: THE DATA CLEANING ENGINE

This is the heart of DataSoul. Three modules, 2,262 lines of code, organized in a strict hierarchy:

```

Layer 1: data_cleaners.py (783 lines) -- Pure Python, zero dependencies beyond Pandas
|
v
Layer 2: llm_cleaner.py (598 lines) -- LLM discovers patterns, Python executes
|
v
Layer 3: csv_corrector.py (881 lines) -- Full orchestrator combining both layers

```

Chapter 16: Layer 1 -- Deterministic Cleaners (data_cleaners.py)

This module is the foundation of DataSoul's cleaning capability. Every function is a `@staticmethod` that takes a Pandas Series and returns a tuple: (cleaned_series, audit_dict_or_None). This design makes each cleaner:

- * Testable: No instance state, no side effects, pure input/output.
- * Composable: Run them in any order, chain them together.
- * Auditable: Every transformation is documented in the audit dict.
- * Independent: No LLM, no network, no external services.

The Null String Constant Set

```

NULL_STRINGS = {
    "n/a", "na", "null", "none", "nil", "nan", "undefined",
    "-", "---", "----", ".", "..", "...",
    "#n/a", "#na", "#null", "#ref!", "#value!",
    "not available", "not applicable", "missing",
    "n.a.", "n.a", "n/d", "nd",
    "unknown",
    "", " ",
}

```

This set contains 24 patterns that humans use to represent "no data." Each pattern has a story:

- * "n/a", "na": The most common null indicators in survey data.
- * "-", "---", "----": Used in financial reports and printed tables.
- * ".", "..", "...": Placeholder indicators from OCR and old databases.
- * "#n/a", "#ref!", "#value!": Excel error codes that leak into CSV exports.
- * "not available", "missing": Verbose null indicators from form submissions.

* "", " ": Empty strings and whitespace-only cells that Pandas does not recognize as NaN.

Cleaner 1: `clean\_null\_strings()`

```
@staticmethod
def clean_null_strings(series, col_name="", extra_nulls=None):
    if not pd.api.types.is_object_dtype(series):
        return series, None    # Only applies to string columns

    null_set = NULL_STRINGS | (extra_nulls or set())
    mask = series.notna()
    str_vals = series[mask].astype(str).str.strip()
    is_null_string = str_vals.str.lower().isin(null_set)
    count = int(is_null_string.sum())

    if count == 0:
        return series, None    # Nothing to clean

    out = series.copy()
    out.loc[str_vals[is_null_string].index] = np.nan

    return out, {
        "action": "CLEAN_NULL_STRINGS",
        "column": col_name,
        "detail": f"Converted {count} null-like strings to NaN",
        "rows_affected": count,
        "confidence": 96,
        "source": "deterministic",
    }
```

This is always the FIRST cleaner to run. Why? Because all downstream cleaners need to know where the actual data is. If a column contains "N/A" strings, the `clean_percentages()` cleaner would try to strip "%" from "N/A" and fail silently. By converting null strings to NaN first, downstream cleaners only see real values.

Cleaner 2: `clean\_whitespace()`

```
@staticmethod
def clean_whitespace(series, col_name=""):
    cleaned = original.copy()
    cleaned = cleaned.str.replace('\u200b', '', regex=False) # zero-width space
    cleaned = cleaned.str.replace('\u200c', '', regex=False) # zero-width non-joiner
    cleaned = cleaned.str.replace('\u200d', '', regex=False) # zero-width joiner
    cleaned = cleaned.str.replace('\uffff', '', regex=False) # BOM
    cleaned = cleaned.str.replace('\xa0', ' ', regex=False) # non-breaking space
    cleaned = cleaned.str.replace(r'[\t\r\n]+', ' ', regex=True)
    cleaned = cleaned.str.replace(r'\s{2,}', ' ', regex=True)
    cleaned = cleaned.str.strip()
```

Why invisible characters are dangerous: A column containing "New York" and "New\u00A0York" (with a non-breaking space) will show as two different categories in value counts, even though they look identical to the human eye. The zero-width characters (\u200b, \u200c, \u200d) are even worse -- they are completely invisible but change string

equality comparisons. The BOM (\uffeff) is a byte-order mark that sometimes appears at the start of UTF-8 files.

Cleaner 3: `clean\_text\_prefixes()`

```
_KNOWN_PREFIXES = [
    "written by:", "writtenby:", "written by",
    "narrated by:", "narratedby:", "narrated by",
    "translated by:", "translatedby:",
    "author:", "authors:",
    "narrator:", "narrators:",
    "publisher:", "published by:",
    "director:", "directed by:",
]
```

This cleaner was born from a real-world dataset: an Audible audiobook catalog where the Author column contained values like "Writtenby:J.K. Rowling" and the Narrator column had "Narratedby:Stephen Fry". The web scraping that created the CSV failed to separate labels from values. The cleaner only triggers when 30% or more of values carry a known prefix, preventing false positives on columns that legitimately contain sentences.

Cleaner 4: `clean\_rating\_strings()`

```
rating_re = re.compile(
    r'^([0-9]+(?:\.[0-9]+)?)\s+out\s+of\s+[0-9]+\s+stars?',
    re.IGNORECASE,
)
```

Converts "4.5 out of 5 stars 234 ratings" to 4.5 (a float). The regex captures only the leading numeric value and ignores everything after "stars." Only triggers when 40% or more values match this pattern.

Cleaner 5: `clean\_booleans()`

```
BOOL_TRUE = {"yes", "y", "true", "t", "1", "on", "active", "enabled", "si", "oui", "à¹à³%à²?"}
BOOL_FALSE = {"no", "n", "false", "f", "0", "off", "inactive", "disabled", "non", "à²"à¹à¹EURà²?"}
```

Converts boolean-like strings to actual Python booleans. Note the internationalization: Hindi (à¹à³%à²?/à²"à¹à¹EURà²?), French (oui/non), and Spanish (si) are supported. The 70% threshold prevents false positives on columns that happen to contain "yes" as a categorical value among many others.

Cleaner 6: `clean\_negative\_parens()`

Accounting software exports negative numbers as (500) instead of -500. This cleaner converts:

- * (500) to -500
- * (\$1,234.56) to -1234.56
- * (â?¹1,00,000) to -100000

The regex handles optional currency symbols and comma-separators. Only triggers when at least 2 values match and they represent more than 5% of the column.

Cleaner 7: `clean\_emails()`

```
domain_fixes = {
  "gmial.com": "gmail.com",    "gmaill.com": "gmail.com",
  "gamil.com": "gmail.com",    "gmai.com": "gmail.com",
  "gmail.co": "gmail.com",     "yaho.com": "yahoo.com",
  "hotmal.com": "hotmail.com", "outlok.com": "outlook.com",
}
```

Only runs on columns whose name contains "email" or "mail." It lowercases all addresses and corrects 10 common domain typos. This is a perfect example of the column-name hinting pattern used throughout DataSoul: cleaners use the column name as a signal for what type of data they are dealing with.

Cleaner 8: `clean\_indian\_numbers()`

Indian number notation groups digits differently from Western notation:

- * Indian: 1,23,45,678 (groups of 2 after the first 3)
- * Western: 12,345,678 (groups of 3)

The cleaner distinguishes between the two using regex patterns. It only applies Indian cleaning when the Indian pattern matches significantly MORE values than the Western pattern, avoiding ambiguous cases like 1,234 which is valid in both systems.

The `run\_all()` Orchestrator

```

@classmethod
def run_all(cls, df, column_hints=None):
    cleaner_sequence = [
        ("null_strings", cls.clean_null_strings),    # FIRST: establish NaN
        ("whitespace", cls.clean_whitespace),        # SECOND: normalize text
        ("text_prefixes", cls.clean_text_prefixes),  # Content-aware
        ("rating_strings", cls.clean_rating_strings), # Content-aware
        ("percentages", cls.clean_percentages),      # Type coercion
        ("booleans", cls.clean_booleans),            # Type coercion
        ("negative_parens", cls.clean_negative_parens), # Type coercion
        ("emails", cls.clean_emails),                # Domain-specific
        ("phone_numbers", cls.clean_phone_numbers),  # Domain-specific
        ("indian_numbers", cls.clean_indian_numbers), # Regional
        ("years", cls.clean_years),                  # Temporal
        ("dates", cls.normalize_dates),              # Temporal
    ]

    for col in list(df_out.columns):
        for cleaner_name, cleaner_fn in cleaner_sequence:
            # Skip string cleaners if column was already converted to numeric
            if not pd.api.types.is_object_dtype(df_out[col]):
                if cleaner_name in string_only_cleaners:
                    continue

            cleaned, audit_entry = cleaner_fn(df_out[col], col_name=col)
            if audit_entry is not None:
                df_out[col] = cleaned
                audit.append(audit_entry)

```

Order matters critically:

- * 1. null_strings first: establishes where real data ends and "missing" begins.
- * 2. whitespace second: normalizes invisible characters before pattern matching.
- * 3. Content-aware cleaners next: strip prefixes and extract ratings while data is still strings.
- * 4. Type coercion cleaners: convert strings to their proper types (float, bool).
- * 5. Domain-specific cleaners: handle emails, phones using column name hints.
- * 6. Temporal cleaners last: dates and years are the most complex and least likely to interfere.

The `is_object_dtype` check prevents a cleaner from running on a column that was already converted to numeric by a previous cleaner. For example, if `clean_percentages()` converted "45%" to 45.0, `clean_booleans()` should not try to run on that column.

Chapter 17: Layer 2 -- LLM-Guided Cleaning (llm_cleaner.py)

The Discovery-Execution Pattern

The LLMCleaner does NOT clean data. It discovers WHAT needs cleaning and returns a plan. Python then

Prompt Engineering: The Batch Column Analysis Prompt

```
For each issue found, return a JSON object:
{{
  "Column1": [
    {{
      "issue": "short_name",
      "fix_type": "strip_prefix|strip_suffix|extract_number|...",
      "confidence": 0.9,
      "prefix_patterns": ["list"]
    }}
  ],
  "Column2": []
}}
```

- * 2. Sample values with frequencies: The prompt includes (x42) counts so the LLM can distinguish common patterns from rare anomalies.
- * 3. RAG context injection: Domain-specific cleaning rules from the knowledge base are included.
- * 4. Explicit pattern list: The LLM is reminded of specific patterns to look for, reducing hallucination.
- * 5. Structured JSON output: The LLM returns a deterministic plan, not free-text advice.

The `clean\_async()` Flow

```
async def clean_async(self, df, min_confidence=0.75):
    df_out = df.copy()
    audit = []

    # Step 1: Infer semantic types for all columns (one LLM call)
    schema = await self._infer_schema_async(df_out)

    # Step 2: Analyze columns in batches of 5
    cols = df_out.columns.tolist()
    tasks = []
    for i in range(0, len(cols), 5):
        batch_cols = cols[i:i+5]
        tasks.append(self._analyse_columns_batch_async(df_out, batch_cols, schema))

    results = await asyncio.gather(*tasks, return_exceptions=True)

    # Step 3: Apply plans with confidence gating
    for col, plans in all_plans.items():
        for plan in plans:
            if plan.get("confidence", 0) < min_confidence:
                continue # Skip low-confidence plans
            entry = self._apply_plan(df_out, col, plan)
            if entry:
                audit.append(entry)

    return df_out, audit
```

The `asyncio.gather(*tasks)` call is crucial for performance. If a dataset has 20 columns, it creates 4 batches of 5. All 4 batches are sent to Ollama concurrently. On a machine with enough GPU memory, all 4 complete faster than running them sequentially.

Plan Execution: The 10 Fix Types

Each `fix_type` in the LLM's plan maps to a deterministic Python implementation:

- * 1. `strip_prefix`: Remove label prefixes like "Author:" from values.
- * 2. `strip_suffix`: Remove unit suffixes like "GB" via regex.
- * 3. `extract_number`: Pull numeric values from strings like "1.37kg" -> 1.37.
- * 4. `to_numeric`: Strip currency/comma characters and convert to float.
- * 5. `to_null`: Convert specific strings to NaN.
- * 6. `regex_replace`: General regex find-and-replace.

- * 7. map_values: Map specific values to new values (case-insensitive).
- * 8. to_boolean: Convert boolean-like strings using provided true/false value lists.
- * 9. normalise_case: Apply title/lower/upper case normalization.
- * 10. deduplicate_words: Remove consecutive duplicate words ("English English" -> "English").

Example: The `_fix_extract_number()` implementation:

```
@staticmethod
def _fix_extract_number(df, col, plan, conf, issue, desc):
    pattern = plan.get("extract_regex", "")
    rx = re.compile(pattern, re.IGNORECASE)

    def extract(v):
        m = rx.search(v)
        return m.group(1) if m else v

    extracted = original.apply(extract)

    # Verify extracted values are actually numeric
    numeric_test = pd.to_numeric(extracted, errors="coerce")
    changed_mask = (extracted != original) & numeric_test.notna()

    # Only apply where extraction succeeded AND result is numeric
    result = original.copy()
    result[changed_mask] = numeric_test[changed_mask]
    df.loc[non_null, col] = result
```

Notice the safety check: the method verifies that extracted values are actually numeric before applying them. If the LLM's regex accidentally matches non-numeric text, those rows are left unchanged. This is the "trust but verify" pattern.

Chapter 18: Layer 3 -- The CSV Corrector (csv_corrector.py)

The CSVCorrector is the most comprehensive cleaning module. It combines deterministic and LLM-guided approaches across 881 lines.

The Mojibake Map

Mojibake is what happens when text is decoded with the wrong character encoding. For example, the UTF-8 sequence for an em-dash (--) is three bytes: 0xE2 0x80 0x94. If these bytes are decoded as Latin-1 instead, you get Ã¢â".

```

MOJIBAKE_MAP = {
    "Ã¢â?¬â?¢": "'", "Ã¢â?¬Â": "'", "Ã¢â?¬\x9d": "'",
    "Ã¢â?¬": "âEUR", "Ã¢â?¬": "âEUR", "Ã¢â?¬!": "âEUR!",
    "Ã?Â©": "Â©", "Ã?Â": "Â", "Ã?Â¼": "Â¼",
    "Ã?Â¶": "Â¶", "Ã?Â": "Â", "Ã?Â±": "Â±",
    "\x00": "", "\uffff": "",
}

```

The CSVCorrector scans every string column for these patterns and replaces them with the correct Unicode characters. This is a purely deterministic operation with 95% confidence.

The `auto\_correct()` Pipeline

The full auto-correction runs 8 stages in strict order:

```

def auto_correct(self, df, profile=None, threshold=None):
    df_out = df.copy()
    audit = []

    # Stage 1: Clean column headers (encoding + whitespace)
    audit.extend(self._clean_column_names(df_out))

    # Stage 2: Fix encoding artifacts in all string columns
    audit.extend(self._fix_encoding(df_out))

    # Stage 3: Run DataCleaners (null strings, booleans, etc.)
    audit.extend(self._fix_semantic_issues(df_out))

    # Stage 4: Fix type issues (currency, references, numeric conversion)
    audit.extend(self._fix_type_issues(df_out))

    # Stage 5: Standardize formats (mixed date/number formats)
    audit.extend(self._fix_formats(df_out, profile))

    # Stage 6: Merge duplicate categories (case + fuzzy + LLM)
    audit.extend(self._merge_categories(df_out, profile, threshold))

    # Stage 7: LLM-guided value corrections
    audit.extend(self._correct_values(df_out, profile, threshold))

    # Stage 8: Drop empty reference/citation columns
    audit.extend(self._drop_empty_reference_columns(df_out))

    return {"status": "success", "df": df_out, "audit": audit, ...}

```

Category Merging -- Three-Pass Algorithm

This is one of the most sophisticated algorithms in DataSoul. Given a column with categories like:

```
"New York", "new york", "NEW YORK", "New York", "Nwe York", "NYC"
```

The corrector runs three passes:

Pass 1 -- Exact Case Match (Deterministic): Group values by lowercase version. "New York" / "new york" / "NEW YORK" all map to the same key. The variant with the highest frequency becomes canonical.

```
lower_map = {}
for val in uniques:
    lower_map.setdefault(val.strip().lower(), []).append(val)
# "new york" -> ["New York", "new york", "NEW YORK"]
# Keep "New York" (highest count)
```

Pass 2 -- Fuzzy Match (Deterministic): Use SequenceMatcher with token-sort ratio to catch typos:

```
def _token_sort(s):
    return " ".join(sorted(s.split()))
# "J.K. Rowling" -> "J.K. Rowling" (same)
# "Rowling, J.K." -> "J.K. Rowling," (very similar after sort)
```

The fuzzy matching includes:

- * Noise prefix stripping ("The ", "Dr. ", "Mr. ")
- * Noise suffix stripping (" Inc", " Ltd", " LLC")
- * Edit distance hard cap: reject if >4 characters differ even if ratio passes

Pass 3 -- LLM Semantic Match: Send the remaining unique values to the LLM for semantic analysis:

```
prompt = f"""Analyze these values from column "{col}" and identify duplicates to merge.
VALUES:
- "NYC"
- "New York City"
- "New York"
Return JSON: [{{"old": "NYC", "new": "New York City", "reason": "abbreviation"}}]"""
```

The LLM can identify that "NYC" is an abbreviation of "New York City" -- something no fuzzy-matching algorithm would catch.

Type Issue Detection and Correction

The `_fix_type_issues()` method handles three categories:

Currency strings: Strips \$, â?¹, ÂGBP, â?¬, ÂY and comma separators, then converts to numeric. Only applies when >30% of values contain currency symbols AND >60% convert successfully.

Embedded references: Removes Wikipedia-style annotations like [1], [a], [17] from values. These appear in datasets scraped from web tables.

Numeric-as-string: When >70% of a string column's values are parseable as numbers, the entire column is

converted to numeric type.

Semantic Issue Detection

The `_detect_semantic_issues()` method identifies:

- * Null-like strings: Using the same `NULL_STRINGS` set from `data_cleaners.py`.
- * Percentage strings: Values like "45%" that should be numeric.
- * Boolean strings: Values like "Yes"/"No" that should be True/False.
- * Negative parentheses: Accounting notation like "(500)" meaning -500.

These are then fixed by delegating to the corresponding `DataCleaners` methods, creating a clean separation between detection and execution.

PART VI: THE ITERATIVE PIPELINE --

pipeline_engine.py

Chapter 19: The Pipeline Philosophy

The IterativePipeline is based on a simple observation: cleaning reveals hidden issues. When you fix encoding in a column, you might discover that the decoded values contain null strings. When you convert strings to numeric, you might reveal new outliers. The pipeline addresses this by running the profiler BEFORE and AFTER cleaning, comparing the results.

```
+-----+ +-----+ +-----+
| Profile |----->| Clean |----->| Profile |
| (Before) |      | (Multi- |      | (After) |
+-----+ | Stage) | +-----+
          +-----+
          |
          |
+-----> Compare Quality Scores <-----+
          |
          Delta Report
          "Completeness: 78 -> 94 (+16)"
          "Consistency: 65 -> 89 (+24)"
```

The Pipeline Class (810 lines)

```
class IterativePipeline:
    def __init__(self, df):
        self._df = df.copy()
        self._profiler = DataProfiler
        self._llm = get_llm()
        self._rag = get_rag()
        self._audit = []
```

The `run()` Method

The main pipeline orchestrates these stages:

```

def run(self):
    # Stage 1: Initial profiling
    initial_profile = DataProfiler(self._df).generate_full_profile()
    initial_threats = ThreatDetector(self._df, initial_profile).detect_all_threats()
    initial_score = initial_profile["quality_score"]["overall"]

    # Stage 2: Deterministic cleaning
    self._df, det_audit = DataCleaners.run_all(self._df)
    self._audit.extend(det_audit)

    # Stage 3: LLM-guided cleaning (if available)
    if self._llm and self._llm.is_available:
        cleaner = LLMCleaner(self._llm)
        self._df, llm_audit = cleaner.clean(self._df)
        self._audit.extend(llm_audit)

    # Stage 4: CSV correction (encoding, types, categories)
    corrector = CSVCorrector()
    result = corrector.auto_correct(self._df, profile=initial_profile)
    self._df = result["df"]
    self._audit.extend(result["audit"])

    # Stage 5: Re-profile to measure improvement
    final_profile = DataProfiler(self._df).generate_full_profile()
    final_threats = ThreatDetector(self._df, final_profile).detect_all_threats()
    final_score = final_profile["quality_score"]["overall"]

    # Stage 6: Generate delta report
    delta = self._compute_delta(initial_profile, final_profile)

    return {
        "df": self._df,
        "initial_score": initial_score,
        "final_score": final_score,
        "improvement": final_score - initial_score,
        "delta": delta,
        "audit": self._audit,
        "initial_profile": initial_profile,
        "final_profile": final_profile,
        "final_threats": final_threats,
    }

```

Quality Delta Analysis

The `_compute_delta()` method compares each quality dimension before and after:

```
def _compute_delta(self, before_profile, after_profile):
    before_dims = before_profile["quality_score"]["dimensions"]
    after_dims = after_profile["quality_score"]["dimensions"]

    delta = {}
    for dim_name in before_dims:
        before_score = before_dims[dim_name]["score"]
        after_score = after_dims[dim_name]["score"]
        delta[dim_name] = {
            "before": before_score,
            "after": after_score,
            "change": round(after_score - before_score, 1),
            "improved": after_score > before_score,
        }
    return delta
```

A typical delta report might look like:

Dimension	Before	After	Change
Completeness	78.2	94.1	+15.9
Consistency	65.0	89.3	+24.3
Type Correctness	45.0	92.0	+47.0
Uniqueness	85.0	95.0	+10.0
Validity	88.0	91.0	+3.0
Accuracy	100.0	100.0	0.0
Timeliness	90.0	90.0	0.0

The `improve()` Method

The targeted improvement method focuses on specific columns:

```
def improve(self, target_col=None):
    if target_col is None:
        # Find the worst-performing column
        profile = DataProfiler(self._df).generate_full_profile()
        target_col = self._find_worst_column(profile)

    # Run targeted cleaning strategies
    # ... specific cleaning for the identified issues ...
```

This is used by the /improve API endpoint to iteratively improve one column at a time, allowing the user to see incremental progress.

PART VII: THE INTELLIGENCE LAYER

Chapter 20: The Data Profiler (profiler.py, 344 lines)

The 7-Dimension Quality Score

The profiler's most important output is the quality score, computed across seven dimensions:

1. Completeness (20% weight):

```
missing_pct = df.isna().sum().sum() / max(total_cells, 1)
completeness = max(0, 100 - (missing_pct * 100 * 2.5))
```

A 10% missing rate yields a completeness score of 75 (100 - 10*2.5). The 2.5x multiplier makes the penalty steep -- missing data is the most impactful quality issue.

2. Uniqueness (15% weight):

```
dup_pct = df.duplicated().sum() / max(rows, 1)
uniqueness = max(0, 100 - (dup_pct * 100 * 5))
```

A 5% duplication rate yields 75. The 5x multiplier is even steeper -- duplicates directly inflate metrics.

3. Consistency (15% weight): Penalizes case inconsistency and trailing whitespace in string columns. If a column has 10 unique values but only 7 unique when lowercased, that 30% fragmentation incurs a penalty.

4. Type Correctness (15% weight): Detects numeric values stored as strings, embedded references, and unparsed currency symbols. Each type of violation carries a different penalty weight.

5. Validity (10% weight): Measures extreme outliers using the 3x IQR method (more conservative than the standard 1.5x IQR). Values beyond Q1-3IQR or Q3+3IQR are considered extreme.

6. Accuracy (10% weight): Penalizes near-zero variance (std < 0.001) and constant-value columns. These indicate columns with no analytical value.

7. Timeliness (15% weight): For date columns, checks recency. If the most recent date is more than 2 years old, the timeliness score decreases, signaling potentially stale data.

The overall score is a weighted sum:

```
overall = (completeness * 0.20 + uniqueness * 0.15 + consistency * 0.15 +
           type_correctness * 0.15 + validity * 0.10 + accuracy * 0.10 +
           timeliness * 0.15)
```

Grading: A+ (>=95), A (>=90), B (>=80), C (>=60), D (>=40), F (<40).

Chapter 21: The Threat Detector (threat_detector.py, 308 lines)

The ThreatDetector scans the profiling results and identifies actionable risks across 8 categories:

Category 1: Missing Value Threats

```
if is_id_column and count > 0:
    severity = "critical" # ID columns must NEVER have missing values
elif pct > 30:
    severity = "critical" # >30% missing = column may be useless
elif pct > 5:
    severity = "warning" # >5% missing = needs attention
else:
    severity = "low" # Minor missing values
```

Each threat includes actionable recommendations:

- * Critical ID missing: "Investigate data pipeline for ingestion failures"
- * >30% missing: "Consider dropping column if non-critical"
- * >5% missing: "Use median for skewed numeric data, mean for normal"

Category 7: Privacy Threats

```
PII_PATTERNS = {
    "email": (r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", "Email Addresses"),
    "phone": (r"\+?\d{1,3}[-.\s]?(\d{3}\s)?[-.\s]?d{3}[-.\s]?d{4}", "Phone Numbers"),
    "ssn": (r"\b\d{3}-\d{2}-\d{4}\b", "SSN/National IDs"),
    "credit_card": (r"\b(?:\d[ -]*){13,16}\b", "Credit Card Numbers"),
}
```

If >30% of sampled values in a column match a PII pattern, a CRITICAL privacy threat is raised. This is particularly important for GDPR and CCPA compliance.

Cleanlab Integration

The threat detector optionally integrates with Cleanlab's OutOfDistribution detection:

```
try:
    from cleanlab.outlier import OutOfDistribution
    ood = OutOfDistribution()
    scores = ood.fit_score(features=clean_vals.values.reshape(-1, 1))
    ood_mask = scores < 0.15 # Score < 0.15 = out-of-distribution
except ImportError:
    pass # Cleanlab not installed -- skip this check
```

This uses machine learning (not just statistics) to identify anomalous values that IQR-based methods might miss.

Chapter 22: The Prediction Engine (prediction_engine.py, 428 lines)

ML-Powered Missing Value Prediction

```
def predict_missing(self, df, target_col, profile=None):
    # Select feature columns (numeric, >50% non-null)
    feature_cols = [c for c in df.select_dtypes(include=[np.number]).columns
                     if c != target_col and df[c].notna().sum() > len(df) * 0.5]

    if is_numeric:
        model = RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=-1)
    else:
        model = RandomForestClassifier(n_estimators=50, random_state=42, n_jobs=-1)

    model.fit(X_train, y_train)
    predictions = model.predict(X_predict)
```

The engine automatically selects the correct model type (regressor vs classifier) based on the target column's data type. Feature importance is extracted from the RandomForest and returned to the user.

Anomaly Detection with Isolation Forest

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=42)
labels = iso.fit_predict(values)
anomaly_mask = labels == -1
```

The contamination=0.05 parameter tells the model to expect ~5% of values to be anomalous. This is a sensible default for most datasets.

LLM Interpretation Layer

Every prediction result is optionally enhanced with LLM interpretation:

```
def _explain_predictions(self, result, profile):
    prompt = f"""Explain these ML predictions briefly (2-3 sentences):
    Column: {result['column']} ({result['type']})
    Model: {result['model']} (train score: {result.get('train_score')})
    Top features: {result.get('feature_importance', {})}"""

    return self._llm.generate(prompt, temperature=0.3, max_tokens=200)
```

This transforms raw ML output into human-readable explanations: "The model predicts missing salary values using tenure and department as key drivers, achieving 89% training accuracy. This suggests a strong relationship between seniority and compensation."

Chapter 23: The Narrative Engine (narrative_engine.py, 422 lines)

Executive Narrative Generation

The `generate_story()` method produces two types of reports:

LLM Mode (when Ollama is available): A free-form narrative generated by the LLM with full dataset context, profile data, threat intelligence, and RAG-retrieved domain knowledge.

Template Mode (fallback): A structured markdown report with sections:

- * 1. Headline: dataset size, health score, threat count.
- * 2. Key Financial Metrics: auto-detected revenue/price columns.
- * 3. Category Breakdown: top categories in categorical columns.
- * 4. Threat Intelligence Summary: critical and warning threats.
- * 5. Data Quality Dimensions: visual bar chart using Unicode blocks.
- * 6. Recommended Next Steps: prioritized action plan.
- * 7. ROI of Analysis: estimated analyst hours saved.

Indian Currency Formatting

```
@staticmethod
def _format_currency(value):
    if abs(value) >= 1_00_00_000:          # 1 crore
        return f"â?¹{value / 1_00_00_000:.2f}Cr"
    elif abs(value) >= 1_00_000:           # 1 lakh
        return f"â?¹{value / 1_00_000:.2f}L"
    elif abs(value) >= 1000:
        return f"â?¹{value / 1000:.1f}K"
    else:
        return f"â?¹{value:.2f}"
```

This reflects DataSoul's Indian origin: large numbers are formatted in lakhs and crores rather than millions and billions.

Conversational Data Chat

The `answer_question()` method implements a sophisticated pattern-matching system:

```
# 14 pattern categories with keyword detection:
if any(kw in q for kw in ["missing", "null", "empty"]):      # Missing values
if any(kw in q for kw in ["duplicate", "dup"]):             # Duplicates
if any(kw in q for kw in ["shape", "size", "rows"]):        # Dataset shape
if any(kw in q for kw in ["outlier", "extreme"]):           # Outliers
if any(kw in q for kw in ["biggest threat", "fix first"]):   # Priority
if any(kw in q for kw in ["improve", "clean"]):             # Improvement
if any(kw in q for kw in ["health", "quality", "score"]):   # Quality
if any(kw in q for kw in ["summary", "overview"]):          # Overview
if any(kw in q for kw in ["top", "best", "highest"]):       # Rankings
if any(kw in q for kw in ["bottom", "worst", "lowest"]):    # Bottom rankings
if any(kw in q for kw in ["correlat", "relat"]):            # Correlations
```

If the question references a specific column name, the engine returns targeted statistics for that column. For complex questions that do not match any pattern, the LLM is used with full RAG context.

Chapter 24: Sector Detector & Strategy Engine

SectorDetector (130 lines)

Eight industry sectors are defined with weighted column patterns:

```
SECTOR_DEFINITIONS = {
    "retail": {
        "name": "Retail & E-Commerce",
        "columns": ["sku", "product", "price", "quantity", "order", "cart", ...],
        "weight": 1.0,
    },
    "healthcare": {
        "name": "Healthcare",
        "columns": ["patient", "diagnosis", "icd", "medication", ...],
        "weight": 1.2, # Higher weight -- more specific vocabulary
    },
    ...
}
```

Detection uses substring matching: if the column name "patient_id" contains the pattern "patient", it counts as a match for the healthcare sector. Healthcare has a higher weight (1.2) because its vocabulary is more specific -- a column called "product" could appear in many sectors, but "diagnosis" is almost certainly healthcare.

StrategyEngine (220 lines)

Recommends preprocessing strategies per column:

Missing value strategy is chosen based on distribution:

- * Skewness > 1.0: recommend median (robust to outliers)

- * Approximately normal: recommend mean
- * Categorical, low cardinality: recommend mode
- * Categorical, high cardinality: recommend adding "Unknown" category
- * >50% missing: recommend dropping the column

Encoding strategy is chosen based on cardinality:

- * Binary (2 values): Label Encoding
- * Low (≤ 10): One-Hot Encoding
- * Moderate (≤ 30): Target Encoding
- * High (> 30): Hash Encoding

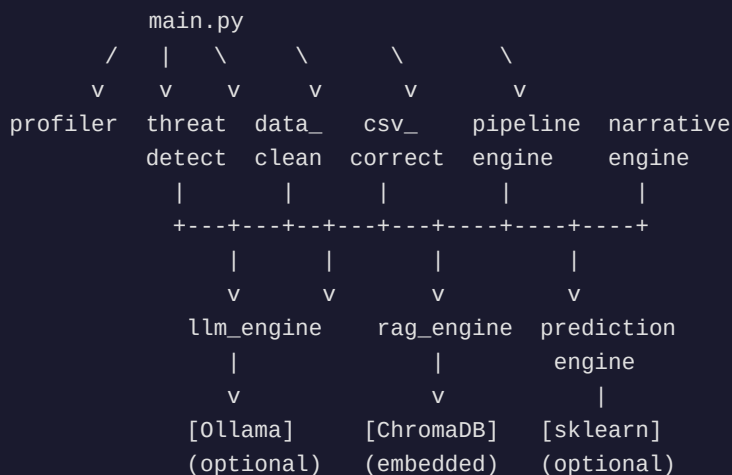
Scaling strategy is chosen based on outliers and distribution:

- * >3% outliers: RobustScaler
- * Right-skewed: Log transform + StandardScaler
- * Normal: StandardScaler

Each recommendation includes a pipeline_step string with the actual scikit-learn code to use.

PART VIII: SYSTEM SYNTHESIS & CONCLUSION

Chapter 25: The Module Dependency Graph



Key observations:

- * profiler.py and data_cleaners.py have ZERO external dependencies (pure Pandas/NumPy).
- * llm_engine.py and rag_engine.py are imported by almost every intelligence module.
- * main.py depends on everything but nothing depends on it (clean facade pattern).

Chapter 26: Design Patterns Catalog

Pattern	Where Used	Purpose
Singleton	<code>`get_llm()`, `get_rag()`</code>	Single shared instance for LLM and RAG
Strategy	<code>`StrategyEngine`</code>	Different algorithms based on data characteristics
Pipeline	<code>`IterativePipeline`</code>	Ordered sequence of cleaning stages
Template Method	<code>`NarrativeEngine`</code>	LLM generation with template fallback
Observer	Audit trail	Side-effect logging of every operation
Facade	<code>`main.py`</code>	Simple API surface for complex backend
Bridge	<code>`LLMEngine._run_async()`</code>	Bridges sync and async execution contexts
Chain of Responsibility	<code>`DataCleaners.run_all()`</code>	Each cleaner handles what it can, passes the rest
Factory Method	<code>`SectorDetector.detect()`</code>	Creates sector-specific context

Chapter 27: Extending DataSoul

Adding a New Deterministic Cleaner

- * 1. Add a new @staticmethod to DataCleaners in data_cleaners.py:

```
@staticmethod
def clean_urls(series, col_name=""):
    # Implementation
    return cleaned_series, audit_dict_or_none
```

- * 2. Add it to the cleaner_sequence in run_all().
- * 3. Add it to _fix_semantic_issues() in csv_corrector.py.

Adding a New LLM Fix Type

- * 1. Add the fix type name to the prompt in _BATCH_COLUMN_ANALYSIS_PROMPT.
- * 2. Add a handler in _apply_plan():

```
elif fix_type == "new_fix_type":
    return self._fix_new_type(df, col, plan, confidence, issue, desc)
```

- * 3. Implement _fix_new_type() as a @staticmethod.

Adding a New Threat Category

- * 1. Add a new _detect_*_threats() method to ThreatDetector.
- * 2. Call it from detect_all_threats().
- * 3. Follow the threat dict structure: id, severity, title, column, category, impact, confidence, actions.

Adding a New Sector

- * 1. Add the sector definition to SECTOR_DEFINITIONS in sector_detector.py.
- * 2. Add sector-specific knowledge to knowledge_scraper.py.

Chapter 28: Glossary

Term	Definition
RAG	Retrieval-Augmented Generation: enriching LLM prompts with r
ChromaDB	An embedded vector database for storing and querying text em
Ollama	A local LLM runtime that serves models via REST API
Mojibake	Garbled text from incorrect character encoding (e.g., "Ã?Â©")
IQR	Interquartile Range: Q3 - Q1, used for outlier detection
PII	Personally Identifiable Information: data that can identify

HIPAA	Health Insurance Portability and Accountability Act (US heal
GDPR	General Data Protection Regulation (EU data privacy law)
Embedding	A numerical vector representation of text for similarity com
Cosine Similarity	A measure of angle between vectors; used for semantic text s
Isolation Forest	An ML algorithm that detects anomalies by isolating outlier
SMOTE	Synthetic Minority Over-sampling Technique for handling clas
OEE	Overall Equipment Effectiveness (manufacturing metric)
CLV	Customer Lifetime Value (retail metric)
RFM	Recency-Frequency-Monetary analysis for customer segmentatio
CORS	Cross-Origin Resource Sharing: browser security mechanism fo
ASGI	Asynchronous Server Gateway Interface: Python async web serv
Winsorization	Capping extreme values at a percentile threshold instead of
VIF	Variance Inflation Factor: detects multicollinearity (>10 is
BOM	Byte Order Mark: invisible character at the start of some te
NBSP	Non-Breaking Space: invisible whitespace character that brea

Conclusion: Lessons from a Vibecoded Architecture

DataSoul AI demonstrates that rapid, AI-assisted development can produce architecturally sound systems when guided by clear principles:

- * 1. Separation of Concerns: Each module has a single responsibility. The profiler profiles. The cleaner cleans. The narrator narrates. No module tries to do everything.
- * 2. Graceful Degradation: The system works at every level of capability, from full AI-powered mode down to basic Pandas operations. This is achieved through consistent is_available checks and fallback paths.
- * 3. Auditability: Every transformation is documented. This is not just good practice -- it is essential for trust. When a user sees that DataSoul changed 42 values in the "Status" column with 96% confidence, they can make an informed decision about whether to keep the change.
- * 4. Hybrid Intelligence: The LLM handles discovery (pattern recognition, semantic understanding) while Python handles execution (vectorized operations, deterministic consistency). This division of labor plays to each technology's strengths.
- * 5. Domain Awareness: Through the RAG knowledge base and sector detection, DataSoul adapts its behavior to the user's domain. A healthcare dataset gets HIPAA-aware threat detection. A retail dataset gets CLV-relevant insights.

The codebase spans approximately 7,500 lines across 14 modules, touching nearly every major area of modern Python development: async web frameworks, database integration, machine learning, natural language processing, vector databases, and data engineering. It serves as a comprehensive case study for building production-grade, AI-augmented data applications.
